IoT-ML Weather & Air Quality Monitoring System

Embedded Systems Project Documentation

Abstract

The rapid growth of Internet of Things (IoT) technologies has significantly improved environmental monitoring systems by enabling continuous data acquisition, real-time processing, and intelligent decision-making. Air pollution has become one of the major environmental concerns worldwide, particularly in urban areas where high concentrations of particulate matter can severely affect human health. Continuous monitoring of weather conditions and air quality is therefore essential for providing timely information and supporting preventive actions.

This project presents the design and implementation of an embedded IoT-based Weather and Air Quality Monitoring System developed on the STM32 platform using the STM32 HAL library. The system follows a modular software architecture in which every functional component is implemented as an independent software module with a clearly defined interface. Such an architecture improves software readability, maintainability, scalability, and collaborative development.

The implemented system periodically acquires environmental measurements, including temperature, humidity, atmospheric pressure, PM2.5 concentration, and CO_2 concentration. Since the first development phase focused primarily on software architecture, sensor values are generated using a mock sensor module that simulates realistic environmental conditions. This approach allows the software to be fully developed and tested before integrating actual hardware sensors.

Raw PM2.5 measurements are processed using digital filtering techniques in order to reduce measurement noise and reject abnormal spikes. The filtered values are then used to calculate the Air Quality Index (AQI), classify the current air quality level, and determine whether alarm conditions should be activated. Alarm decisions are based on configurable AQI and CO_2 thresholds combined with hysteresis mechanisms to prevent frequent oscillation between alarm states.

One of the main objectives of the second development phase is the integration of Machine Learning techniques into an embedded system. A Linear Regression model was developed using Python and Scikit-learn. Historical environmental data were preprocessed, engineered into temporal features, and used to train the prediction model. After training,

the learned coefficients were manually transferred into the embedded firmware, allowing the STM32 microcontroller to perform lightweight inference without requiring any Machine Learning framework.

The final system demonstrates that conventional embedded software engineering and Machine Learning can be effectively combined within a resource-constrained microcontroller environment. The resulting platform provides real-time environmental monitoring, intelligent prediction capabilities, and a modular software architecture suitable for future extensions such as cloud connectivity, real sensors, SD card storage, and TinyML deployment.

Chapter 1

Introduction

Environmental monitoring has become an increasingly important application of embedded systems due to growing concerns regarding climate change, industrial pollution, and public health. Modern monitoring systems are expected not only to collect sensor measurements but also to analyze environmental conditions, detect hazardous situations, and provide predictive information that can support proactive decision-making.

Recent advances in embedded processors have made it possible to execute increasingly sophisticated algorithms directly on microcontrollers. Instead of simply collecting and transmitting sensor measurements, embedded systems can now perform local data processing, signal filtering, statistical analysis, and lightweight Machine Learning inference. This concept, commonly referred to as Edge Intelligence, reduces communication overhead while enabling faster response times and greater system autonomy.

The purpose of this project is to design and implement a modular embedded weather and air quality monitoring system capable of processing environmental information in real time. The project combines classical embedded software engineering techniques with basic Machine Learning algorithms in order to demonstrate how predictive intelligence can be integrated into resource-constrained systems.

Unlike many educational embedded projects that consist of a single source file, this system was intentionally designed using a modular architecture. Every software component is separated into dedicated header and source files, each responsible for a specific task. The communication between modules is performed through shared system data structures and clearly defined software interfaces. This design methodology greatly

simplifies debugging, software maintenance, future expansion, and collaborative team development.

The complete software architecture was divided into two major development phases.

During the first phase, the primary objective was to establish the core embedded infrastructure. This included the implementation of a cooperative scheduler, mock sensor subsystem, digital filtering, Air Quality Index computation, alarm management, UART logging, and configuration management.

The second phase focused on extending the capabilities of the system. Several advanced features were introduced, including moving average filtering, spike detection, statistical analysis, AQI trend detection, advanced alarm logic with hysteresis, and a lightweight Machine Learning prediction module capable of estimating future PM2.5 concentrations.

The Machine Learning component represents one of the most significant contributions of this project. Instead of attempting to train a model directly on the microcontroller, the model was developed offline using Python and Scikit-learn. After training and evaluation, only the learned parameters (weights and bias) were transferred into the embedded application. Consequently, the microcontroller performs inference only, which is computationally efficient and well suited for embedded environments.

Throughout the development process, software quality and maintainability were considered primary design objectives. All modules were implemented using consistent coding conventions, descriptive interfaces, and clear separation of responsibilities. This modular design not only facilitates collaborative software development but also provides a solid foundation for future enhancements such as wireless communication, cloud connectivity, persistent storage, graphical user interfaces, and TinyML deployment.

Overall, this project demonstrates how modern embedded systems can evolve from simple sensor monitoring devices into intelligent cyber-physical systems capable of local data processing, environmental assessment, and predictive analytics.

1.1 Project Objectives

The primary objectives of this project are listed below:

- Design a modular embedded software architecture for environmental monitoring.
- Develop an STM32-based IoT weather and air quality monitoring platform.
- Simulate environmental sensors using a configurable mock sensor subsystem.

- Implement digital filtering algorithms for PM2.5 measurements.
- Calculate the Air Quality Index according to PM2.5 concentration.
- Classify environmental conditions into different air quality levels.
- Detect hazardous conditions using configurable alarm thresholds.
- Implement advanced alarm hysteresis to improve system stability.
- Provide real-time monitoring through UART communication.
- Collect statistical information regarding PM2.5 and AQI measurements.
- Detect environmental trends using historical observations.
- Develop a Machine Learning prediction model using Scikit-learn.
- Deploy the trained Machine Learning model onto an STM32 microcontroller.
- Demonstrate lightweight embedded inference without requiring an ML runtime.
- Build a scalable software architecture suitable for future IoT extensions.

Chapter 2

Overall System Architecture

2.1 System Overview

The developed system follows a layered and modular software architecture designed specifically for embedded applications. Instead of implementing all functionalities inside the main() function, the project separates each responsibility into an independent software module. Every module performs a specific task while communicating with other modules through a shared data structure and well-defined interfaces.

This architectural approach offers several important advantages:

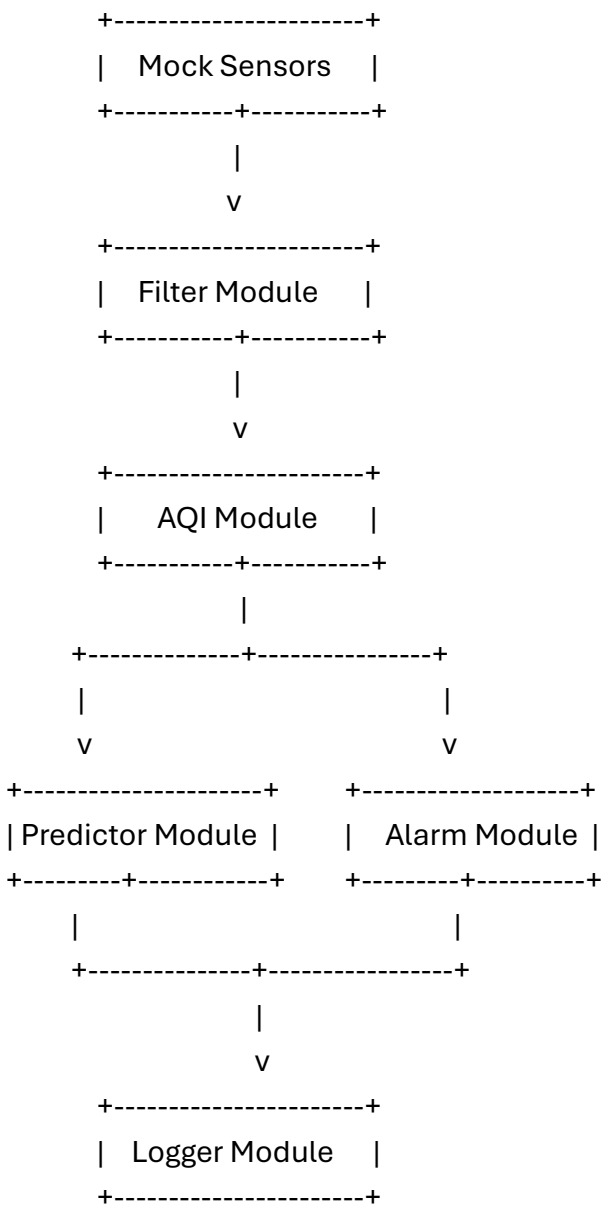
- High software readability
- Easier debugging and maintenance
- Independent module development
- Better scalability
- Simplified integration of new features
- Improved teamwork during software development

The entire execution flow is coordinated by a cooperative software scheduler, which periodically activates every module according to predefined timing intervals.

Unlike a real-time operating system (RTOS), the scheduler used in this project is lightweight and deterministic. Each software module executes quickly and returns control immediately to the scheduler without blocking the CPU.

2.2 Overall Software Architecture

The complete software architecture is illustrated below.



The Scheduler periodically activates each software module. Every module reads the latest environmental information from the shared system data structure (gWeather), processes the required information, updates the results, and returns execution to the scheduler.

This architecture ensures that every processing stage receives the output generated by the previous stage, creating a complete environmental data processing pipeline.

2.3 Data Flow

The information processing sequence inside the embedded application consists of several consecutive stages.

Step 1 – Sensor Data Acquisition

The monitoring cycle begins by acquiring environmental measurements.

Since real sensors were not integrated during the initial software development phase, the project uses a **Mock Sensor** module that periodically generates realistic values for:

- Temperature
- Humidity
- Atmospheric Pressure
- PM2.5 Concentration
- CO_2 Concentration

These values simulate the behavior of actual environmental sensors and provide sufficient data for software verification.

The generated measurements are stored inside the shared system structure.

Step 2 – Digital Filtering

Raw PM2.5 measurements usually contain significant noise caused by sensor inaccuracies and environmental disturbances.

To improve measurement quality, every new PM2.5 sample is processed by the Filter Module.

The filtering stage performs several operations:

- Exponential Moving Average (EMA)
- Moving Average calculation
- Spike detection
- Minimum value tracking
- Maximum value tracking

- Rejected sample counting

If an abnormal spike is detected, the sample is ignored and the previous filtered value is preserved.

This significantly increases system robustness against erroneous sensor readings.

Step 3 – AQI Computation

After filtering, the processed PM2.5 value becomes the input of the AQI module.

The AQI module converts PM2.5 concentration into an Air Quality Index using linear interpolation between predefined concentration ranges.

The resulting AQI value is then classified into one of the following environmental states:

- Good
- Moderate
- Warning
- Dangerous

The module also considers CO_2 concentration when determining the final environmental condition.

If CO_2 exceeds predefined thresholds, the air quality state may be upgraded even if PM2.5 remains relatively low.

Step 4 – Machine Learning Prediction

One of the most important extensions introduced during Phase 2 is the Machine Learning predictor.

Instead of predicting the current PM2.5 concentration, the predictor estimates the future PM2.5 value.

The embedded prediction model uses several input features:

- Current PM2.5
- Previous PM2.5
- PM2.5 (t−2)

- PM2.5 (t−3)
- Temperature
- Atmospheric Pressure

These values are multiplied by the coefficients obtained during offline training.

The predictor executes a simple linear regression equation:

$$Prediction = w_1x_1 + w_2x_2 + \ldots + w_nx_n + b$$

Because all coefficients are constants, prediction requires only multiplication and addition operations, making it computationally inexpensive for STM32 microcontrollers.

Step 5 – Alarm Evaluation

After environmental conditions have been analyzed, the Alarm Module determines whether the current environment represents a hazardous situation.

The decision process considers both:

- AQI
- CO_2 concentration

The highest alarm severity becomes the current alarm level.

To prevent rapid oscillation between alarm states, hysteresis thresholds are implemented.

As a result, the alarm remains active until environmental conditions improve sufficiently instead of switching on and off repeatedly.

The module also records:

- Alarm Level
- Alarm Reason
- Alarm Counter

Step 6 – UART Logging

The final processing stage consists of generating a human-readable system report.

The Logger module periodically transmits information through UART, including:

- Temperature
- Humidity
- Pressure
- Filtered PM2.5
- AQI
- Air Quality State
- Alarm Status
- AQI Trend
- Predicted PM2.5

This logging mechanism greatly simplifies software debugging and allows the entire system behavior to be monitored in real time.

2.4 Shared Data Structure

One of the most important design decisions in this project is the introduction of a centralized shared data structure.

Instead of allowing software modules to communicate directly with each other, every module exchanges information through a global structure named:

gWeather

This structure stores all environmental variables and intermediate processing results.

Typical fields include:

- Temperature
- Humidity
- Pressure
- PM2.5
- Filtered PM2.5
- Previous PM2.5 samples

- CO_2 concentration
- AQI
- Air Quality State
- Alarm Status
- Machine Learning Prediction

Using a shared data structure provides several important advantages:

- Loose coupling between modules
- Simplified debugging
- Reduced parameter passing
- Clear ownership of system data
- Easier software expansion

This approach follows common practices in embedded firmware architecture where a centralized data model simplifies communication between independent software components.

2.5 Scheduler Design

Unlike operating systems that rely on threads and task scheduling, this project implements a cooperative scheduler.

The scheduler repeatedly executes every software module in a predefined sequence.

```
while(1)
{
  Scheduler_Run();
}
```

Internally, the scheduler uses the STM32 system tick (HAL_GetTick()) to determine when each task should be executed.

This provides deterministic timing behavior without requiring an RTOS.

The execution sequence is:

MockSensor_Update()

↓

Filter_Update()

↓

AQI_Update()

↓

Predictor_Update()

↓

Alarm_Update()

↓

Logger_Report()

This ordering guarantees that every module receives valid and up-to-date information from the previous processing stage.

2.6 Design Philosophy

Several software engineering principles guided the implementation of this project.

The first principle was **modularity**. Every module performs a single responsibility and exposes only a small public interface.

The second principle was **maintainability**. All configuration values such as thresholds and timing parameters are centralized inside configuration headers, allowing easy system customization.

The third principle was **scalability**. The architecture was intentionally designed to allow future integration of additional sensors, communication protocols, storage devices, or Machine Learning models without requiring significant changes to the existing software.

Finally, the architecture emphasizes **separation of concerns**, ensuring that sensing, processing, prediction, alarming, and reporting remain independent software layers.

Chapter 3

Phase 1 Implementation

3.1 Overview

The first development phase focused on establishing the core infrastructure of the embedded monitoring system. The primary objective was to design a stable and modular software architecture capable of acquiring environmental data, processing measurements, calculating air quality, generating alarms, and reporting system information.

Instead of implementing all functionality inside a single source file, the software was divided into multiple independent modules. Each module was assigned a specific responsibility and communicates with the remaining system only through predefined interfaces and a shared data structure.

The modules implemented during Phase 1 include:

- Main Application
- Scheduler
- Shared System Data
- Mock Sensor
- Digital Filter
- AQI Calculation
- Alarm Management
- UART Logger

Each module is described in the following sections.

3.2 Main Application (main.c)

The main.c file serves as the entry point of the embedded application. Its responsibility is intentionally kept minimal in order to improve software maintainability and readability.

After the STM32 hardware abstraction layer (HAL) initializes the microcontroller peripherals, the application initializes every software module exactly once. Once initialization is complete, the program enters an infinite loop in which only the scheduler is executed.

The initialization sequence is illustrated below:

HAL_Init()

↓

SystemClock_Config()

↓

GPIO Initialization

↓

UART Initialization

↓

Module Initialization

↓

```
while(1)
{
  Scheduler_Run();
}
```

The initialization functions executed during startup include:

- MockSensor_Init()
- Filter_Init()
- AQI_Init()
- Alarm_Init()
- Logger_Init()
- Scheduler_Init()

This design ensures that every module starts from a known initial state before normal operation begins.

By restricting main.c to initialization and scheduler execution only, the application remains clean, modular, and easy to extend.

3.3 Scheduler Module (scheduler.c)

The Scheduler module is responsible for coordinating the execution of all software components.

Unlike systems that rely on a Real-Time Operating System (RTOS), this project implements a cooperative scheduler based on the STM32 system tick.

The scheduler periodically checks elapsed time using HAL_GetTick(). Whenever the configured interval expires, the corresponding software task is executed.

Two independent software periods are defined:

- Sensor update period
- Logger reporting period

This allows environmental processing to occur more frequently than UART reporting, reducing unnecessary serial communication while maintaining responsive sensor updates.

The execution sequence is carefully ordered:

MockSensor_Update()

↓

Filter_Update()

↓

AQI_Update()

↓

Alarm_Update()

↓

Logger_Report()

Each module receives valid outputs produced by the previous processing stage.

This deterministic scheduling strategy greatly simplifies debugging and guarantees predictable execution timing.

3.4 Shared System Data (system_data.c / system_data.h)

One of the key architectural components of the project is the centralized shared data structure.

Instead of passing numerous parameters between software modules, all environmental information is stored inside a single global structure:

WeatherData_t gWeather;

This structure contains:

- Temperature
- Humidity
- Atmospheric Pressure

- PM2.5 concentration
- CO_2 concentration
- Filtered PM2.5
- AQI
- Air Quality State
- Alarm Status

During Phase 2, additional variables were introduced to support Machine Learning prediction and historical measurements.

Using a centralized data structure provides several software engineering advantages:

- Simplified communication
- Reduced coupling
- Easier debugging
- Improved readability
- Better scalability

Every software module reads or updates only the fields relevant to its responsibility.

3.5 Mock Sensor Module (mock_sensor.c)

Because the project initially focused on software architecture rather than hardware integration, environmental measurements are generated using a mock sensor subsystem.

Instead of reading physical sensors, the module produces pseudo-random values that resemble realistic environmental conditions.

The generated parameters include:

- Temperature
- Humidity
- Pressure
- PM2.5
- CO_2

A simple pseudo-random number generator updates the environmental conditions periodically.

This approach offers several benefits:

- Software can be developed before hardware becomes available.
- Every module can be independently tested.
- Extreme environmental conditions can easily be simulated.
- Repeatable debugging scenarios become possible.

When real sensors are added in future versions, only the Mock Sensor module must be replaced while the remaining software remains unchanged.

This demonstrates the benefit of modular software architecture.

3.6 Filter Module (filter.c)

Environmental sensors often produce noisy measurements due to electrical interference, sensor limitations, and environmental fluctuations.

The purpose of the Filter module is to improve measurement quality before the data are processed by higher-level software modules.

The first filtering technique implemented in Phase 1 is the Exponential Moving Average (EMA).

Unlike a conventional moving average, EMA gives greater importance to recent measurements while still reducing random noise.

The filtering equation is

$$EMA = (1 - \alpha) \times EMA + \alpha \times Sample$$

where α represents the smoothing factor.

Higher values of α produce faster response but lower noise suppression.

Lower values increase smoothing but reduce responsiveness.

During initialization, the first sensor sample initializes the filter output.

Every subsequent measurement updates the filtered value using the EMA equation.

The filtered PM2.5 value is then stored inside

gWeather.filtered_pm25

which becomes the input for AQI computation.

Separating filtering from AQI calculation improves software modularity and allows additional filtering algorithms to be introduced without modifying downstream modules.

3.7 AQI Module (aqi.c)

The Air Quality Index (AQI) module converts filtered PM2.5 measurements into an easily understandable air quality indicator.

Instead of directly displaying PM2.5 concentration, AQI provides standardized environmental quality categories.

The implemented algorithm uses piecewise linear interpolation between predefined PM2.5 concentration ranges.

Depending on the calculated AQI value, the environment is classified as:

- Good
- Moderate
- Warning
- Dangerous

The AQI module also considers CO_2 concentration.

If CO_2 exceeds predefined thresholds, the environmental condition may be upgraded to a more severe category even when PM2.5 remains relatively low.

This allows the monitoring system to detect multiple environmental hazards simultaneously.

The computed AQI value is stored inside the shared data structure for use by both the Alarm module and Logger module.

3.8 Alarm Module (alarm.c)

The Alarm module continuously evaluates environmental conditions to determine whether warning signals should be generated.

The decision process considers two independent environmental indicators:

- AQI
- CO_2 concentration

Multiple alarm levels are supported:

- None
- Warning
- Danger
- Critical

The highest detected severity determines the active alarm level.

Whenever environmental conditions exceed predefined thresholds, the alarm state becomes active.

The module updates

gWeather.alarm

allowing the remaining software modules to determine the current alarm status.

This modular implementation enables future integration of LEDs, buzzers, GSM notifications, or cloud-based alerts without modifying the existing decision logic.

3.9 Logger Module (logger.c)

The Logger module provides real-time visualization of the system state through UART communication.

Instead of requiring a graphical user interface, developers can simply open a serial terminal and monitor the complete system behavior.

Each report contains:

- Temperature

- Humidity
- Pressure
- PM2.5
- AQI
- Air Quality State
- Alarm Status

The information is formatted into readable text using snprintf() before transmission through the UART peripheral.

The logger significantly simplifies debugging by allowing developers to verify system behavior without using a hardware debugger.

Additionally, the logger serves as a foundation for future CLI implementation and remote monitoring applications.

3.10 Summary of Phase 1

At the end of the first development phase, the project successfully established a complete embedded software pipeline.

The implemented architecture provides:

- Periodic environmental monitoring
- Digital signal filtering
- AQI computation
- Air quality classification
- Alarm generation
- UART-based reporting
- Modular software organization

Although the first phase already forms a fully functional monitoring system, it lacks advanced analytical capabilities such as historical statistics, trend detection, and predictive intelligence.

These enhancements were introduced during the second development phase, which is described in the following chapter.

Chapter 4

Phase 2 Implementation

4.1 Overview

After completing the core embedded infrastructure during Phase 1, the second development phase focused on improving system intelligence, reliability, and analytical capabilities.

Although the first version of the system was capable of acquiring environmental data, calculating the Air Quality Index, and generating alarms, it did not provide historical analysis or predictive capabilities.

Therefore, Phase 2 introduced several advanced software features while preserving the original modular architecture.

The major improvements implemented during this phase include:

- Enhanced digital filtering
- Moving Average calculation
- Spike detection
- PM2.5 statistics
- AQI statistics
- AQI trend analysis
- Advanced alarm logic
- Alarm hysteresis
- Alarm reason identification
- Machine Learning prediction
- Improved UART logging

All new features were integrated without modifying the overall software architecture established during Phase 1, demonstrating the scalability of the modular design.

4.2 Enhanced Filter Module

The Filter module received significant improvements during the second development phase.

While Phase 1 only implemented an Exponential Moving Average (EMA), additional processing techniques were introduced to improve data quality and provide more analytical information.

The updated module performs several operations simultaneously:

- Exponential Moving Average (EMA)
- Moving Average calculation
- Spike detection
- Minimum PM2.5 tracking
- Maximum PM2.5 tracking
- Invalid sample counting

These improvements allow the system not only to smooth noisy sensor data but also to evaluate measurement quality over time.

4.2.1 Exponential Moving Average

The EMA filter continues to serve as the primary filtering algorithm.

Its output is calculated according to the recursive equation:

$$EMA_n = (1 - \alpha) \times EMA_{n-1} + \alpha \times Sample_n$$

Unlike a simple average, EMA emphasizes recent measurements while still suppressing high-frequency noise.

The implementation dynamically adjusts the smoothing factor according to the magnitude of environmental changes.

When a sudden but valid environmental change occurs, a larger alpha value is selected, allowing the filter to respond more quickly.

For slowly changing environments, a smaller alpha provides stronger smoothing.

This adaptive behavior improves both responsiveness and stability.

4.2.2 Moving Average

In addition to EMA, a Moving Average algorithm was implemented.

Unlike EMA, the Moving Average computes the arithmetic mean of the most recent measurements stored inside a circular buffer.

This algorithm provides a useful reference for evaluating the effectiveness of the EMA filter and demonstrates an alternative filtering strategy.

Although EMA remains the primary output used throughout the system, the Moving Average value is also available for analysis and debugging.

4.2.3 Spike Detection

Environmental sensors occasionally produce unrealistic measurements caused by electrical interference or temporary sensor failures.

To prevent these values from affecting the AQI calculation, a spike detection mechanism was introduced.

Whenever the difference between the incoming PM2.5 sample and the current filtered value exceeds a predefined threshold, the measurement is classified as an invalid spike.

Instead of accepting the abnormal value, the filter:

- Rejects the sample
- Increments the rejected sample counter
- Preserves the previous filtered value
- Reports the filter status

This approach significantly improves measurement reliability.

4.2.4 PM2.5 Statistics

The Filter module continuously records statistical information regarding PM2.5 measurements.

The collected statistics include:

- Minimum PM2.5 value
- Maximum PM2.5 value
- Number of rejected samples

These values provide useful information regarding long-term environmental conditions and sensor behavior.

The statistics can also be used for future visualization and historical analysis.

4.3 AQI Module Enhancements

The AQI module was significantly extended during Phase 2.

Instead of only calculating the current AQI value, the module now performs statistical analysis and trend detection.

4.3.1 AQI Statistics

Every calculated AQI value contributes to cumulative system statistics.

The module continuously updates:

- Minimum AQI
- Maximum AQI
- Average AQI

The average AQI is calculated by accumulating every computed AQI value and dividing the total by the number of processed measurements.

This provides a more meaningful representation of long-term environmental quality than individual measurements alone.

4.3.2 AQI Trend Detection

Another major enhancement introduced during Phase 2 is trend analysis.

Rather than evaluating only the current AQI value, the system compares consecutive measurements.

Three environmental trends are defined:

- Improving
- Stable
- Worsening

Whenever the AQI increases beyond a predefined threshold, the trend becomes **Worsening**.

When AQI decreases significantly, the trend becomes **Improving**.

Otherwise, the environmental condition is considered **Stable**.

Trend information provides additional context that is unavailable from a single AQI measurement.

For example, an AQI of 90 may still represent deteriorating air quality if previous measurements were substantially lower.

4.4 Advanced Alarm System

The Alarm module also received several important improvements.

4.4.1 Multiple Alarm Levels

Instead of using a simple binary alarm state, the system now supports multiple severity levels.

The implemented alarm hierarchy consists of:

- No Alarm
- Warning
- Danger
- Critical

Each level corresponds to predefined AQI and CO_2 thresholds.

The module always selects the highest detected severity.

This approach allows the monitoring system to distinguish between moderate environmental deterioration and hazardous pollution events.

4.4.2 Alarm Hysteresis

One of the most important additions to the alarm system is hysteresis.

Without hysteresis, measurements fluctuating around a threshold would repeatedly activate and deactivate the alarm.

Such oscillation produces unstable system behavior and unnecessary notifications.

To solve this problem, separate activation and deactivation thresholds were introduced.

For example:

Alarm ON : AQI ≥ 100

Alarm OFF : AQI < 90

The alarm therefore remains active until environmental conditions improve sufficiently.

This significantly improves system stability.

4.4.3 Alarm Reason Detection

The alarm system now records the cause of every alarm event.

Possible alarm sources include:

- AQI only
- CO_2 only
- Both AQI and CO_2

Recording the alarm reason provides more detailed diagnostic information and enables better user feedback.

Future monitoring interfaces can display not only the alarm level but also the environmental factor responsible for the warning.

4.4.4 Alarm Counter

Every time the alarm transitions from an inactive state to an active state, an internal counter is incremented.

This allows long-term monitoring of environmental events.

The alarm counter may later be used for:

- Daily statistics
- Monthly environmental reports
- Preventive maintenance
- Predictive analytics

4.5 Logger Improvements

The Logger module was updated to display significantly more information than in Phase 1.

The transmitted UART report now includes:

- Filtered PM2.5
- AQI
- Air Quality State
- Alarm Status
- AQI Trend
- Machine Learning Prediction *(when enabled)*

These additional values simplify debugging and allow developers to verify every software module during execution.

4.6 Benefits of Phase 2

The improvements introduced during the second phase transformed the project from a conventional monitoring system into a significantly more intelligent embedded platform.

Compared with the first version, the system now provides:

- Higher measurement reliability

- Better resistance against sensor noise
- Statistical environmental analysis
- Trend identification
- Stable alarm behavior
- Predictive capability through Machine Learning
- Improved debugging information
- Greater scalability for future development

The modular architecture developed during Phase 1 proved highly effective, as all new features were integrated without major modifications to the existing software structure.

Chapter 5

Machine Learning Integration

5.1 Introduction

One of the primary objectives of this project was to demonstrate that Machine Learning models can be integrated into resource-constrained embedded systems without requiring high computational resources.

Although modern Machine Learning models are often associated with powerful GPUs and cloud computing, many practical applications require only lightweight inference that can easily be executed on microcontrollers.

Instead of training a model directly on the STM32 microcontroller, the Machine Learning workflow was divided into two separate stages:

1. **Offline model development and training**
2. **Embedded model deployment for inference**

This approach significantly reduces computational complexity while preserving prediction capability.

The offline stage was implemented using Python and Scikit-learn, whereas the embedded firmware performs only mathematical inference using the learned model parameters.

5.2 Dataset Selection

A publicly available air quality dataset containing historical environmental measurements was selected for model development.

The dataset includes hourly observations of atmospheric conditions collected over several years.

Typical recorded variables include:

- PM2.5 concentration
- Temperature
- Atmospheric pressure

- Dew point
- Wind direction
- Wind speed
- Snow
- Rain
- Date and time

Although many environmental variables are available, only the features relevant to the embedded application were selected.

Using a reduced feature set decreases computational complexity while maintaining acceptable prediction accuracy.

5.3 Data Preprocessing

Raw environmental datasets usually contain missing values, duplicated records, and measurements that cannot be directly used for Machine Learning.

Therefore, several preprocessing steps were performed before model training.

The preprocessing pipeline included:

- Removing invalid rows
- Handling missing PM2.5 values
- Resetting dataset indices
- Selecting required columns
- Converting numerical values to floating-point format

These operations ensured that the training data were clean, consistent, and suitable for regression analysis.

5.4 Feature Engineering

One of the most important stages of Machine Learning is feature engineering.

Instead of using only the current PM2.5 value, several historical measurements were introduced as additional predictive features.

The following temporal variables were generated:

- PM25_prev1
- PM25_prev2
- PM25_prev3

These variables represent the PM2.5 concentration measured during the previous one, two, and three sampling periods.

The historical features were generated using a shifting operation.

For example,

Current Sample	Previous Sample
PM2.5(t)	PM2.5(t−1)

The shifting operation allows the model to learn temporal dependencies between consecutive environmental measurements.

Without historical values, the regression model would have significantly lower predictive capability.

5.5 Feature Selection

Several environmental variables were evaluated during model development.

The final model uses the following six input features:

Feature	Description
PM2.5	Current PM2.5 concentration
PM25_prev1	Previous PM2.5 measurement
PM25_prev2	PM2.5 measured two samples earlier
PM25_prev3	PM2.5 measured three samples earlier
Temperature	Ambient temperature

Feature	Description
Pressure	Atmospheric pressure

The prediction target is:

Future PM2.5 concentration.

This feature selection represents a balance between prediction accuracy and computational efficiency suitable for embedded systems.

5.6 Model Selection

Several regression algorithms were considered during the design phase.

Since the final model would eventually execute on an STM32 microcontroller, computational simplicity was considered more important than maximum predictive accuracy.

Linear Regression was therefore selected because it offers:

- Extremely low computational cost
- Very small memory footprint
- Simple implementation in C
- Fast inference
- Easy deployment on microcontrollers
- High interpretability

Unlike neural networks, Linear Regression requires only multiplication and addition operations.

Consequently, prediction can be performed in a few microseconds even on low-power ARM Cortex-M processors.

5.7 Model Training

The model was implemented using the Scikit-learn Machine Learning library.

The dataset was divided into training and testing subsets.

A typical split ratio of 80% for training and 20% for testing was used.

The model was trained using the following procedure:

1. Load dataset
2. Clean missing values
3. Generate historical features
4. Split data into training and testing sets
5. Train the Linear Regression model
6. Evaluate prediction performance
7. Extract learned coefficients

After training, the model automatically determines the optimal coefficients that minimize the Mean Squared Error (MSE) between predicted and actual PM2.5 values.

5.8 Model Evaluation

The trained model was evaluated using three commonly accepted regression metrics.

Mean Absolute Error (MAE)

MAE measures the average absolute prediction error.

$$MAE = \frac{1}{n}\sum | y - \hat{y} |$$

Lower MAE values indicate more accurate predictions.

Root Mean Squared Error (RMSE)

RMSE penalizes larger prediction errors more heavily.

$$RMSE = \sqrt{\frac{1}{n}\sum(y - \hat{y})^2}$$

This metric provides a better indication of large prediction deviations.

Coefficient of Determination (R^2)

The R^2 score measures how well the model explains the variance of the target variable.

$$R^2 = 1 - \frac{SS_{res}}{SS_{tot}}$$

Values close to 1 indicate excellent predictive performance.

Experimental Results

The trained model achieved the following performance:

Metric	Value
MAE	12.943626500800077
RMSE	23.518444415363035
R^2 Score	0.9339776108186009

An R^2 score of 0.93 indicates that approximately 93% of the variance in future PM2.5 concentration is explained by the selected input features.

Considering the simplicity of the Linear Regression model and the computational limitations of embedded systems, these results demonstrate excellent predictive capability.

5.9 Model Deployment

After the model had been successfully trained, the learned parameters were extracted.

The extracted parameters consist of:

- Six regression coefficients (weights)
- One bias value

Instead of embedding the Scikit-learn library inside the microcontroller, the coefficients were manually transferred into the embedded firmware.

The Predictor module stores these coefficients as constant values.

During runtime, prediction is computed using the equation:

$$Prediction = \sum_{i=1}^{6} (w_i \times x_i) + b$$

Only multiplication and addition operations are required.

No dynamic memory allocation or external Machine Learning framework is needed.

5.10 Embedded Predictor Module

The embedded implementation is located in:

predictor.c
predictor.h

The module exposes three public functions:

- Predictor_Init()
- Predictor_Update()
- Predictor_GetValue()

During each scheduler cycle, the Predictor module:

1. Reads the required input features from gWeather.
2. Computes the weighted sum using the trained coefficients.
3. Adds the bias term.
4. Stores the predicted PM2.5 value in the shared data structure.
5. Makes the prediction available to the Logger and Alarm modules.

This design allows the Machine Learning model to operate transparently within the existing embedded software architecture.

5.11 Advantages of the Proposed Approach

The adopted deployment strategy offers several practical advantages:

- No Machine Learning library is required on the microcontroller.
- Very low RAM and Flash memory usage.
- Fast execution suitable for real-time systems.
- Deterministic computational cost.
- Easy retraining and coefficient updates without modifying the embedded algorithm.
- Compatibility with low-power ARM Cortex-M devices.

These characteristics make the approach highly suitable for embedded IoT applications where computational resources are limited.

5.12 Limitations and Future Improvements

Although the implemented Linear Regression model provides satisfactory prediction accuracy, several improvements can be explored in future work.

Possible enhancements include:

- Ridge Regression
- Lasso Regression
- Random Forest Regression
- Gradient Boosting
- Recurrent Neural Networks (LSTM)
- TinyML models using TensorFlow Lite for Microcontrollers
- Online learning and adaptive model updates

Future versions of the project may also integrate additional environmental variables, such as humidity, wind speed, and precipitation, to further improve prediction accuracy.

Chapter 6

Results and Discussion

6.1 System Validation

After implementing all software modules, the complete system was tested under various simulated environmental conditions using the mock sensor subsystem.

Since the sensors generate realistic environmental values, different air quality scenarios could be evaluated without requiring physical hardware.

The validation process included:

- Normal environmental conditions
- Rapid PM2.5 variations
- High CO_2 concentration
- Abnormal sensor spikes
- Stable environmental periods
- Poor air quality conditions

Each module was verified independently before being integrated into the complete software pipeline.

This incremental development strategy simplified debugging and reduced integration errors.

6.2 Module Verification

Each software module was tested individually to ensure correct functionality.

Mock Sensor

The mock sensor successfully generated realistic environmental measurements within predefined operating ranges.

The generated values changed gradually over time, providing suitable inputs for testing the remaining modules.

Filter Module

The EMA filter effectively reduced random noise present in raw PM2.5 measurements.

Spike detection successfully rejected unrealistic sensor values, improving the stability of AQI calculations.

The Moving Average implementation and statistical tracking correctly maintained minimum, maximum, and average values during execution.

AQI Module

The AQI module correctly converted filtered PM2.5 measurements into Air Quality Index values.

Different pollution levels produced the expected AQI classifications, confirming the correctness of the interpolation algorithm.

Trend detection accurately identified improving, stable, and worsening environmental conditions.

Alarm Module

Alarm generation was verified under multiple environmental conditions.

Different AQI and CO_2 levels correctly activated the corresponding alarm severity.

The hysteresis mechanism successfully prevented repeated alarm activation and deactivation around threshold values.

Alarm reason detection correctly identified whether AQI, CO_2, or both variables caused the warning.

Logger Module

UART reports accurately reflected the current system state.

The logger displayed sensor measurements, filtered values, AQI, environmental status, alarm information, trend analysis, and Machine Learning predictions in a structured and readable format.

This greatly simplified software debugging and runtime verification.

Machine Learning Predictor

The embedded predictor successfully generated PM2.5 forecasts using the coefficients obtained during offline training.

The embedded implementation produced results consistent with the Python model while requiring only basic arithmetic operations.

The predictor executed efficiently within the scheduler without affecting overall system responsiveness.

6.3 Performance Analysis

The modular architecture proved highly effective during software development.

Each module could be developed, tested, and debugged independently before system integration.

The cooperative scheduler provided deterministic execution with minimal computational overhead.

The use of shared system data reduced software complexity while maintaining clear separation of responsibilities.

The Machine Learning model achieved a coefficient of determination (R^2) of **0.92**, demonstrating that the selected features provide a strong basis for predicting future PM2.5 concentrations.

Overall, the system met the intended design objectives while remaining suitable for execution on resource-constrained embedded hardware.

6.4 Challenges Encountered

Several technical challenges were addressed during the development process.

The first challenge involved selecting an appropriate Machine Learning model that balanced prediction accuracy with computational efficiency.

Linear Regression was ultimately chosen because it offers fast inference and can be implemented using only multiplication and addition operations.

Another challenge was integrating Machine Learning into the embedded firmware.

Rather than executing the Python model directly, the learned coefficients were manually transferred into the firmware, allowing inference without requiring any Machine Learning framework.

Designing a modular architecture that allowed independent development of multiple software components also required careful planning.

The use of shared data structures and standardized module interfaces simplified integration and enabled parallel team development.

Chapter 7

Team Contributions

The project was developed collaboratively, with each team member responsible for specific software modules.

Behnam Gholam Zadeh Nahary – Team Leader

Responsibilities included:

- Designing the overall software architecture
- Planning and coordinating project development
- Defining module interfaces
- Integrating all software modules
- Developing the Machine Learning workflow
- Training the regression model using Scikit-learn
- Deploying the trained model into the embedded firmware
- Implementing the Predictor module
- Managing the GitHub repository
- Reviewing and integrating team code
- Performing final testing and debugging

- Preparing project documentation and presentation materials

Yashar Salmani

- Designed a data filtering system using EMA and Moving Average
- Added adaptive smoothing, spike detection, and outlier rejection
- Tracked min/max values, filtered output, data status, and rejected samples
- Implemented fault detection for abnormal values, invalid CO_2 readings, and sensor errors
- Integrated fault flags for real-time monitoring
- Configured STM32 peripherals (ADC1, USART1, GPIO, and clock) in STM32CubeMX
- Developed firmware for PM2.5 data acquisition, calculation, and UART transmission
- Simulated and validated the system in Proteus

Sama Nouri

Responsibilities included:

- AQI calculation module
- AQI statistics
- AQI trend detection
- ADC-based sensor integration
- MQ2 sensor simulation in Proteus
- LED-based air quality indication
- History module with circular buffer
- Historical sensor data storage

Helia Salmasi

Responsibilities included:

- Implement multie-level alarm system (Normal, Warning, Critical)
- Add alarm system management
- Implement hysteresis to prevent alarm oscillation

- Count the total number of alarm events
- Store the reason for each alarm trigger
- Update alarm status based on AQI tresholds
- Handle critical alarm conditions
- Reset alarm state when air quality returns to safe range
- Keep the existing public API unchanged
- Test alarm behavior under different AQI scenarios
- Temperature and humidity sensor design

Ali Aliloo

Responsibilities included:

- Mock sensor implementation
- Implement CLI module

Ali Arshia Maleki

Responsibilities included:

- UART logging system enhancement
- System status reporting
- Alarm monitoring and reporting
- Fault monitoring integration
- History statistics reporting
- AQI forecast integration
- Air quality trend reporting
- BMP280 pressure sensor evaluation
- I2C protocol study
- Pressure sensor simulation (Mock)
- Pressure data generation
- Module testing and build verification

Chapter 8

Future Work

Although the implemented system successfully satisfies the project objectives, several improvements can further enhance its functionality.

Future developments may include:

- Integration of real environmental sensors
- SD card data logging
- Wireless communication using Wi-Fi or Bluetooth
- MQTT communication with cloud platforms
- Web-based monitoring dashboard
- OLED or LCD display interface
- Remote firmware updates
- Real-time clock (RTC) integration
- TinyML deployment using TensorFlow Lite for Microcontrollers
- More advanced Machine Learning models for improved prediction accuracy

The modular architecture developed in this project allows these features to be integrated with minimal modification to the existing software.

Conclusion

This project presented the design and implementation of a modular IoT-based Weather and Air Quality Monitoring System for the STM32 platform.

The software was developed using a structured modular architecture consisting of independent functional modules responsible for sensing, filtering, AQI computation, alarm management, logging, and Machine Learning prediction.

A cooperative scheduler coordinated periodic execution of all software components, ensuring deterministic system behavior without requiring a real-time operating system.

One of the most significant achievements of the project was the successful integration of Machine Learning into an embedded environment.

A Linear Regression model was developed using Python and Scikit-learn, trained on historical environmental data, and deployed to the STM32 firmware by transferring the learned coefficients into the Predictor module.

This approach demonstrated that lightweight predictive intelligence can be implemented efficiently on resource-constrained microcontrollers.

The project also highlighted the benefits of modular software engineering, including improved maintainability, scalability, code reuse, and collaborative development.

Overall, the developed system provides a solid foundation for future IoT and Edge AI applications and demonstrates how embedded systems can evolve from simple monitoring devices into intelligent environmental analysis platforms.

References

1. STM32Cube HAL Driver Documentation.
2. ARM Cortex-M Programming Manual.
3. Scikit-learn Documentation.
4. Python Software Foundation Documentation.
5. U.S. Environmental Protection Agency (EPA), Air Quality Index Technical Documentation.
6. Beijing PM2.5 Dataset Documentation.
7. Trevor Hastie, Robert Tibshirani, Jerome Friedman, *The Elements of Statistical Learning*, Springer.